29. Goroutines and the Scheduler

29. Goroutines and the Scheduler

Phase: Phase 5 — Concurrency
Estimated time: 1–2 days
Slug: `29-goroutines`

Learning objectives

- Start goroutines with go
- Explain M:N scheduling at a high level
- Avoid leaking goroutines
- Synchronize completion with WaitGroup

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Goroutines

```
go func() {
    doWork()
}()
```

Goroutines are lightweight concurrent functions multiplexed onto OS threads by the Go scheduler (M:N).

Leaks

A goroutine leak is a goroutine that never exits — common when blocked on channel/IO forever. Always plan an exit path (context, close channel, timeout).

WaitGroup

```
var wg sync.WaitGroup
wg.Add(1)
go func() {
    defer wg.Done()
    work()
}()
wg.Wait()
```

Do not assume goroutine order. Concurrency ≠ parallelism (but Go gives you both tools).

Common mistakes (especially from Python instincts)

- Fire-and-forget goroutines in request handlers without limits
- Capturing loop variables incorrectly (modern Go fixed per-iteration vars; still be careful)
- Waiting forever with no timeout

Exercises

Work in `exercises/29-goroutines/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/29-goroutines/` until you have an honest failing attempt.

Self-check

1. What does the go keyword do?
2. What is a goroutine leak?
3. Why WaitGroup?

Further reading

- https://go.dev/tour/concurrency
- https://go.dev/blog/scheduler